

Tuple

1. Introduction

A tuple in Python is an **ordered, immutable** collection of items. Tuples can store items of mixed data types and are similar to lists but **cannot be modified** after creation.

2. Definition of a Tuple

A tuple is defined by placing items inside parentheses (), separated by commas.

Example:

```
t1 = (1, 2, 3)
t2 = ("apple", "banana", "cherry")
t3 = (10, "hello", 3.14, True)
```

Single Element Tuple:

```
t = (5,) # valid tuple
t = (5)  # not a tuple, just an integer
```

Empty Tuple:

```
t = () # empty tuple
```

3. Characteristics of Tuples

- Ordered: Elements have a fixed index
- Immutable: Cannot be modified
- Heterogeneous: Can store mixed data types
- Allows duplicates: Repeated values are allowed
- Faster than lists: Better performance for read-only data

4. Accessing Tuple Elements

Accessing tuple elements is similar to list.

Indexing Example:

```
t = (10, 20, 30, 40)
print(t[0]) # 10
print(t[-1]) # 40
```

Slicing:

```
print(t[1:3]) # (20, 30) (similar to list)
```

5. Tuple Methods

Tuples have few methods because they are immutable.

- `count()`:
Returns number of occurrences of an element. (0 if element not found)

```
t = (1, 2, 2, 3)
t.count(2) # Output: 2
```

- `index()`:
Returns first index of an element.

```
t = (10, 20, 30, 20)
t.index(20) # Output: 1
```

This method has some optional parameters.

`tuple.index(element, start = 0, end = len(tuple) )`

- **element**: The value to search for.
- **start** (optional): The starting index for the search.
- **end** (optional): The ending index for the search

If the element is not found in the tuple, `ValueError` is raised.

6. Concatenation and Repetition of Tuples

- Concatenation:

```
t1 = (1,2)
t2 = (3,4)
t3 = t1 + t2 # t3 = (1,2,3,4)
```

- Repetition:

```
t1 = (1,2)
t2 = t1 * 3 # t2 = (1,2,1,2,1,2)
```

7. Packing and Unpacking of Tuples

- Tuple packing:
Putting multiple values together without using parentheses. Python automatically group the values into a tuple

```
t1 = 10 ,20,30  
print(t1) # Output: (10,20,30)
```

- Tuple unpacking:
Assigning elements of the tuple to multiple variables in single statement.

```
t = ( 100 , 200 , 300)  
  
a , b, c = t  
print(a , b , c) # Output = 100 200 300
```